Projeto – Cidades ESGInteligentes

Plataforma de Monitoramento ESG com Pipeline CI/CD, Containerização Docker e Deploy Automatizado

FIAP – Análise e Desenvolvimento de Sistemas

Integrantes:
João Pedro Fernandes Silva – RM561005
Pedro Teixeira Spinola – RM560104
Thiago C. G. Antunes – RM559359

Data: 21/abr/2026
Repositório: https://github.com/lpsgverrilla/fiap-esg-devops

1. Descrição do Projeto

O projeto **Cidades ESGInteligentes** é uma plataforma web de monitoramento de indicadores ESG (Environmental, Social, Governance) construída sobre o trabalho acadêmico anterior do grupo (*Monitoramento de Emissões de Carbono*, MongoDB/NoSQL). A entrega desta disciplina adiciona à aplicação original todo o ciclo DevOps: containerização, orquestração multi-serviço, suíte de testes automatizados, pipeline de CI/CD e deploy automatizado em dois ambientes (staging e produção) em domínios com TLS.

A API REST implementa operações CRUD para as cinco collections ESG (`empresas`, `emissoes`, `impacto_social`, `governanca`, `metas_sustentabilidade`), aproveitando a flexibilidade schema-less do MongoDB — cada empresa registra apenas os indicadores que fazem sentido para seu setor.

Ambientes em produção

Ambiente	URL pública	Branch que dispara deploy	Porta interna
Produção	https://esg.pedrots.com.br	`main`	127.0.0.1:8007
Staging	https://esg-staging.pedrots.com.br	`develop`	127.0.0.1:8006

2. Pipeline CI/CD

Ferramenta

GitHub Actions, utilizando **GitHub Environments** para escopar secrets por ambiente (o `STAGING_DEPLOY_TOKEN` só é injetado em jobs vinculados ao environment `staging`; o `PROD_DEPLOY_TOKEN`, apenas em jobs de `production`).

Workflows

2.1 ci.yml – Integração Contínua

Dispara em **todo push ou pull request em qualquer branch**. Garante que o código nunca regrida antes de chegar às branches protegidas.

- **Job `lint-and-test`**: executa `ruff check`, `ruff format --check` e `pytest` (7 testes cobrindo health, OpenAPI, ciclo CRUD completo em empresas e flexibilidade schema-less em emissões).
- **Job `docker-build`**: usa Buildx com cache GHA para montar a imagem Docker. Falha rápido se o Dockerfile ou dependências quebram. Depende do `lint-and-test`.

2.2 deploy.yml – Entrega Contínua

Dispara em push nas branches `main` e `develop`. Sequência:

1. Job `test`: guard-rail — reexecuta a suíte pytest.
2. Condicionalmente, um dos dois jobs roda:
   - `deploy-staging` (se branch = `develop`): `POST https://esg-staging.pedrots.com.br/hooks/deploy-fiap-esg-staging` com header `X-Deploy-Token`.
   - `deploy-production` (se branch = `main`): `POST https://esg.pedrots.com.br/hooks/deploy-fiap-esg-production`.
3. No VPS, `adnanh/webhook` valida o token e invoca o `deploy.sh` do ambiente:

   ```
   git fetch + checkout branch correta
   docker compose build --pull
   docker compose up -d --remove-orphans
   ```

4. O job espera o endpoint `/health` responder `{"status":"ok"}` por até 150 s antes de marcar verde.

Fluxo de trabalho recomendado

```
feature branch → PR → CI verde → merge em develop
                                  ↓
                        deploy automático em staging
                                  ↓
                        validação manual no ambiente
                                  ↓
            PR develop → main → deploy automático em produção
```

3. Docker – Arquitetura e Estratégia

3.1 Dockerfile (multi-stage)

Imagem Python 3.12-slim em dois estágios:

- **Builder**: instala dependências em prefixo isolado (`/install`), sem sobrepor o site-packages do sistema.
- **Runtime**: copia apenas as dependências instaladas e o código-fonte; executa como usuário não-root `app`; expõe a porta 8000; define `HEALTHCHECK` interno consultando `/health`.

Resultado: imagem final ~180 MB, sem toolchain de build, superfície de ataque mínima.

3.2 docker-compose.yml

Dois serviços em rede Docker dedicada, com nomes parametrizados pelo `APP_ENV` (permite rodar múltiplas instâncias isoladas no mesmo host):

- **`api`** – FastAPI + Uvicorn. Depende do healthcheck do `mongo` estar verde antes de subir.
- **`mongo`** – MongoDB 7, volume nomeado `mongo-data`, healthcheck via `mongosh ping`. O seed inicial é executado automaticamente pela imagem oficial do Mongo ao encontrar o arquivo `/docker-entrypoint-initdb.d/01-seed.js`, populando as 5 collections com 10 documentos cada na primeira boot.

3.3 Arquitetura no VPS

```
Internet ─┬─► nginx :443 (TLS) ─► esg.pedrots.com.br         ─► 127.0.0.1:8007 ─►
api(prod)     ─► mongo(prod)
          └─► nginx :443 (TLS) ─► esg-staging.pedrots.com.br ─► 127.0.0.1:8006 ─►
api(staging) ─► mongo(staging)

                                adnanh/webhook (127.0.0.1:9000)
                                  ├─► /opt/fiap-esg/staging/deploy.sh
                                  └─► /opt/fiap-esg/production/deploy.sh
```

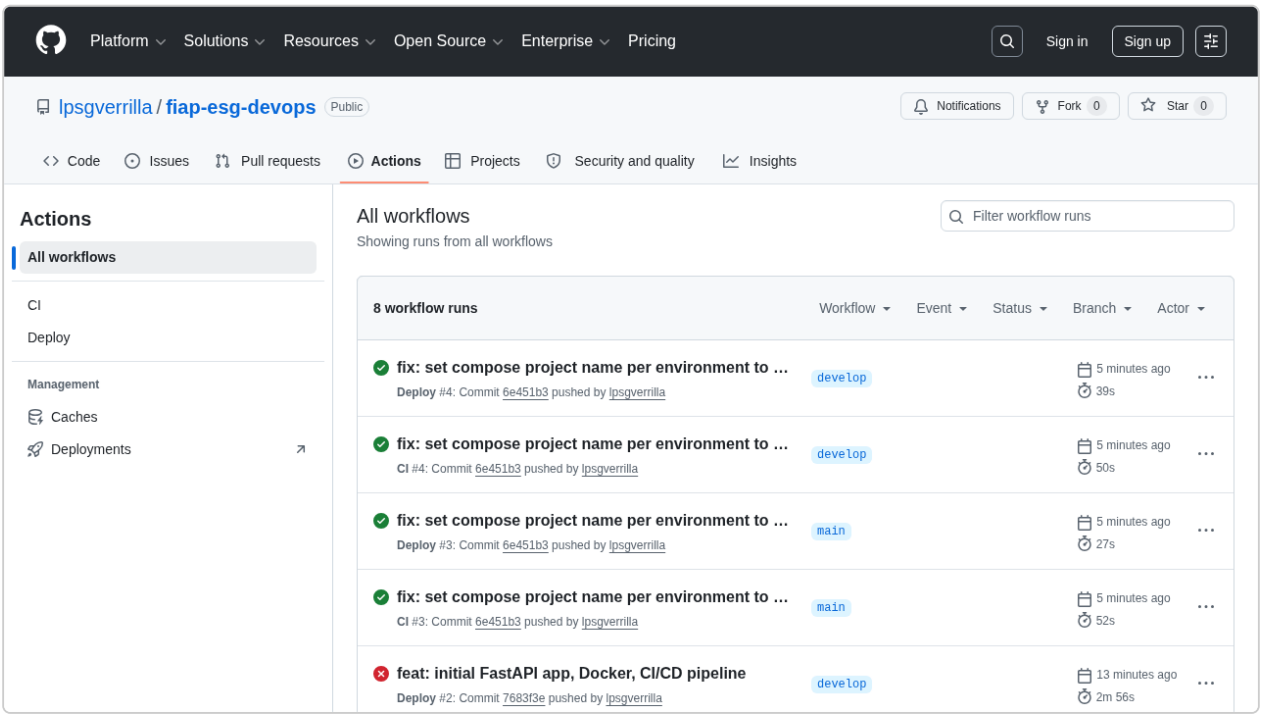
Cada ambiente tem **container, volume e rede Docker isolados** — zero cross-contamination entre staging e produção. O certificado TLS é um SAN emitido pelo Let's Encrypt cobrindo os dois domínios, com auto-renovação via systemd timer.

3.4 Comandos úteis

```
# Subir localmente
docker compose up --build

# Verificar seed
curl localhost:8000/empresas/stats/count    # → {"count":10}

# Derrubar preservando volume
docker compose down

# Derrubar limpando volume (reseed na próxima subida)
docker compose down -v

# Ver estado no VPS
ssh root@vps 'docker ps --filter name=fiap-esg'
```

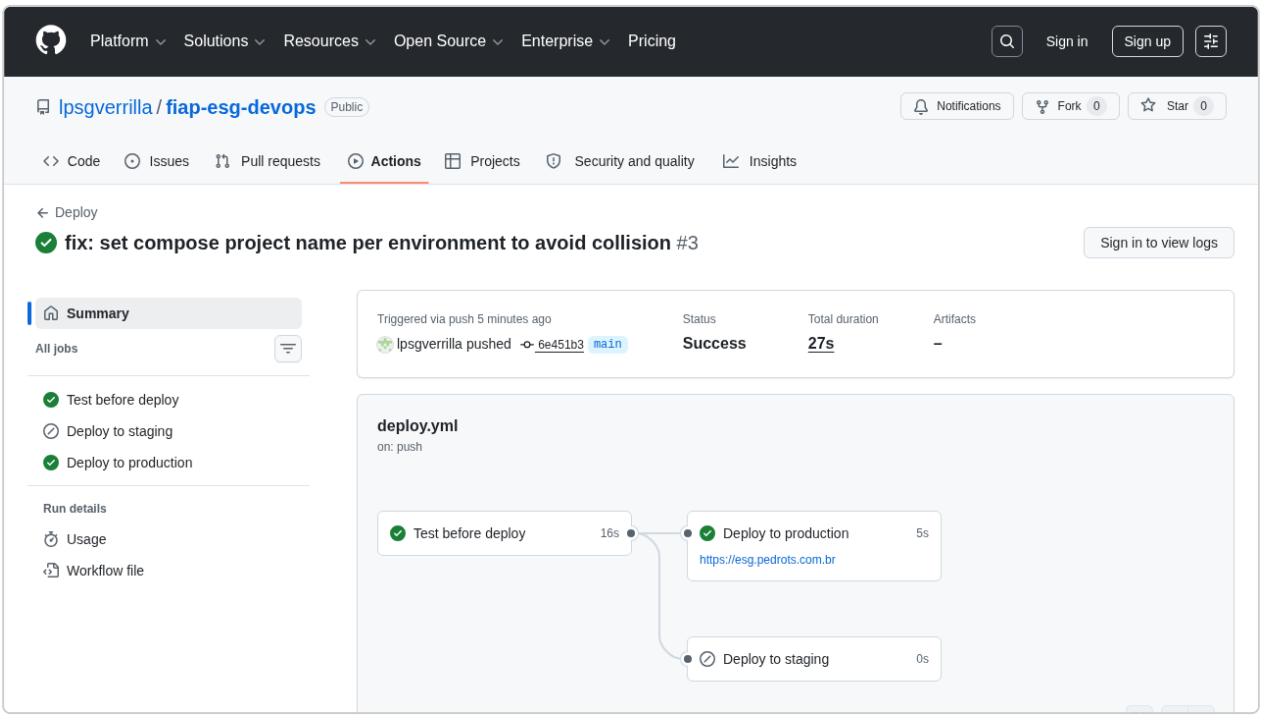
4. Evidências

4.1 Pipeline GitHub Actions – lista de execuções



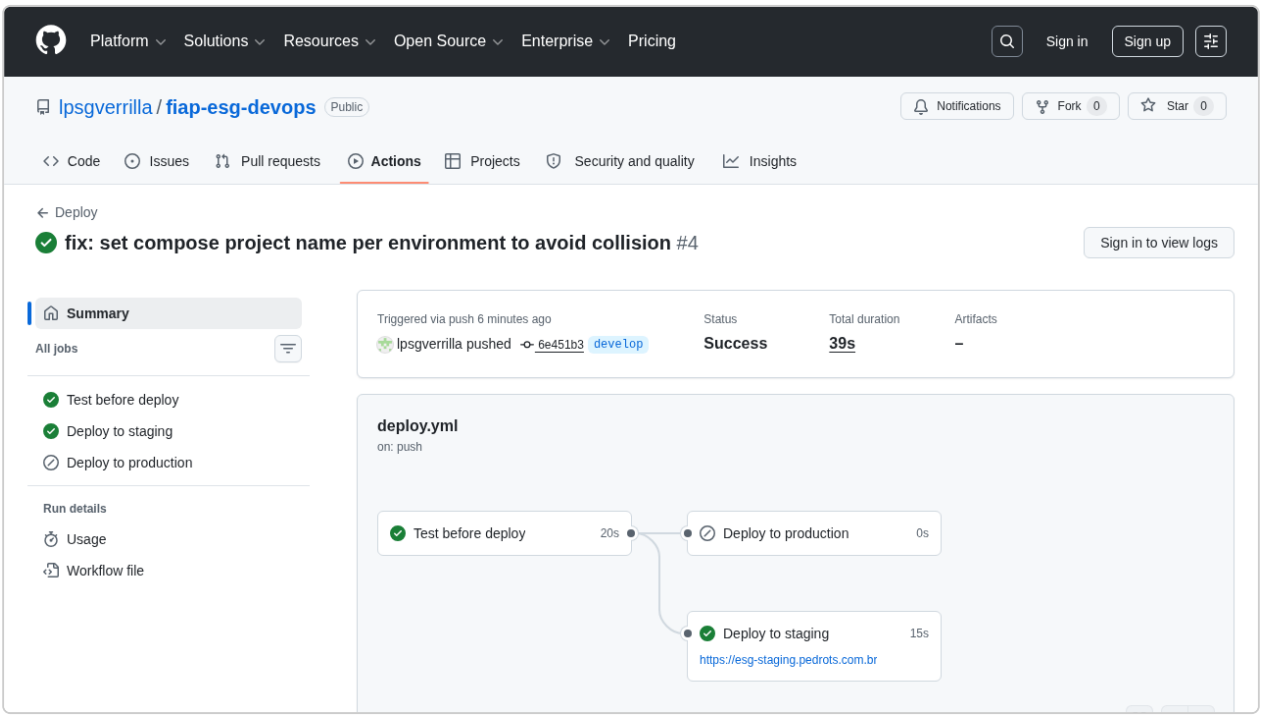
Runs recentes: CI e Deploy verdes nas branches `main` e `develop` após o fix de compose project name.

4.2 Deploy verde em Produção



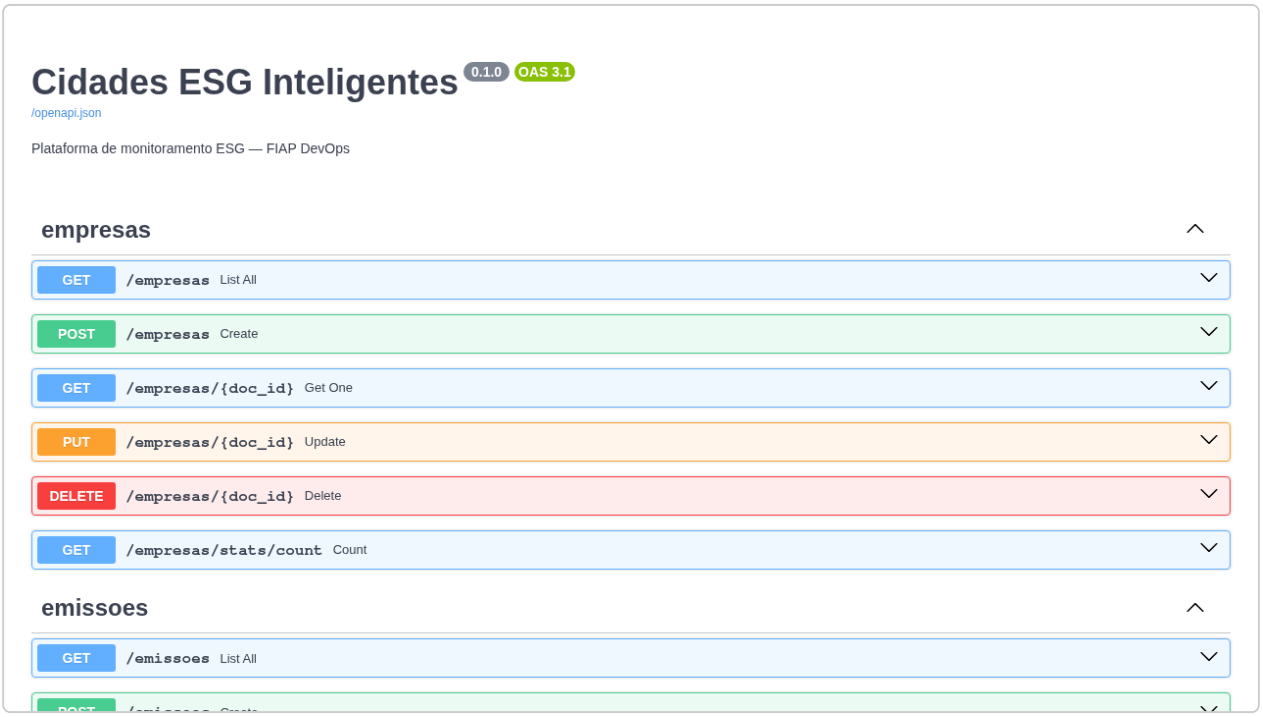
Job `deploy-production` concluído — passa pelo guard-rail de testes, dispara o webhook e confirma o healthcheck em `https://esg.pedrots.com.br/health`.

4.3 Deploy verde em Staging



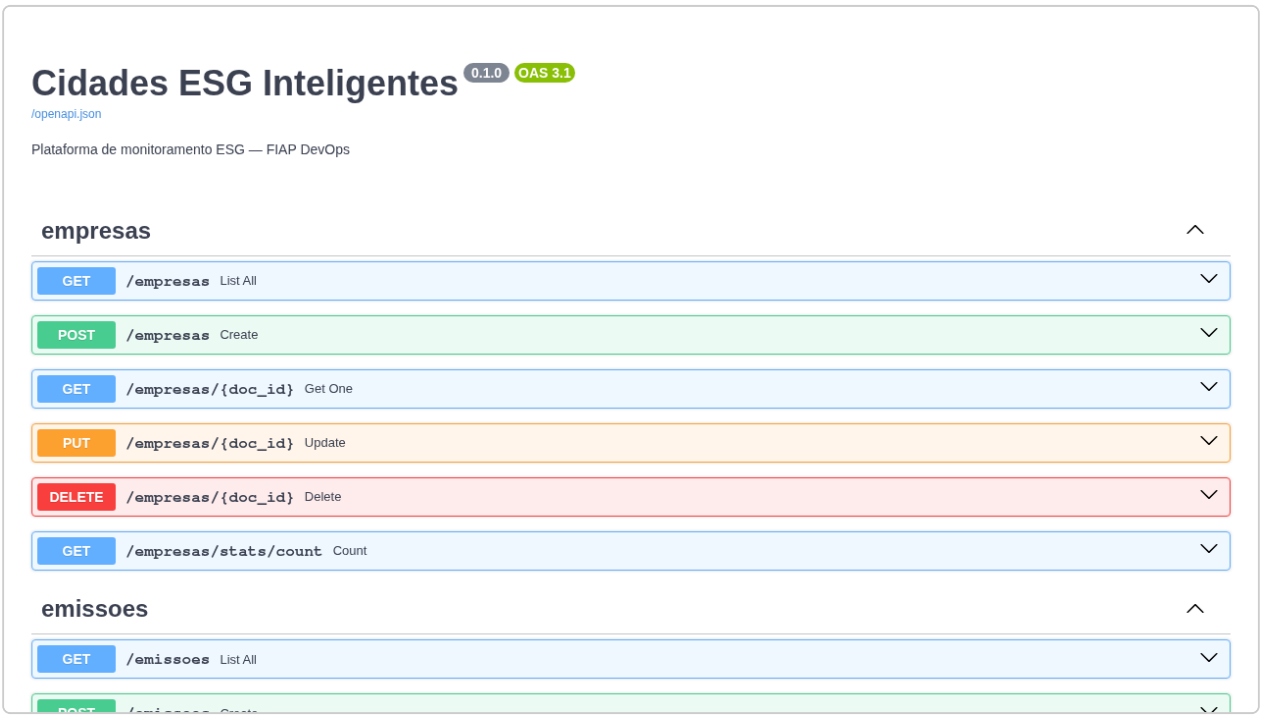
Job `deploy-staging` concluído com mesmo fluxo para `https://esg-staging.pedrots.com.br`.

4.4 API rodando em Produção – Swagger UI



Swagger interativo em `https://esg.pedrots.com.br/docs` listando os endpoints das 5 collections ESG.

4.5 API rodando em Staging – Swagger UI



Mesma documentação servida em `https://esg-staging.pedrots.com.br/docs` *— ambientes independentes.*

4.6 Estado dos containers e reverse proxy no VPS

```
=== docker ps (ambos ambientes) ===
NAMES                       STATUS                    PORTS
fiap-esg-api-staging        Up 5 minutes (healthy)    127.0.0.1:8006->8000/tcp
fiap-esg-mongo-staging      Up 5 minutes (healthy)    27017/tcp
fiap-esg-api-production     Up 8 minutes (healthy)    127.0.0.1:8007->8000/tcp
fiap-esg-mongo-production   Up 9 minutes (healthy)    27017/tcp

=== webhook hooks cadastrados ===
    "id": "deploy-fiap-esg-staging",
    "id": "deploy-fiap-esg-production",

=== nginx sites habilitados ===
fiap-esg-production
fiap-esg-staging

=== certbot cert FIAP ESG ===
  Certificate Name: esg-staging.pedrots.com.br
    Domains: esg-staging.pedrots.com.br esg.pedrots.com.br
    Expiry Date: 2026-07-20 (VALID: 89 days)
```

4.7 Smoke tests pós-deploy

```
$ curl https://esg.pedrots.com.br/health
{"status":"ok","env":"production","db":"ok"}

$ curl https://esg-staging.pedrots.com.br/health
{"status":"ok","env":"staging","db":"ok"}

$ curl https://esg.pedrots.com.br/empresas/stats/count
{"count":10}

$ curl https://esg-staging.pedrots.com.br/empresas/stats/count
{"count":10}

$ curl 'https://esg.pedrots.com.br/emissoes?limit=1'
[{"_id":"69e7e92f6ad955ad8544ba93","empresa":"EcoLog Transportes",
  "ano":2025,"emissao_CO2_ton":1200,"diesel_litros":500000,
  "energia_renovavel_percentual":25}]
```

5. Desafios e soluções

Colisão de *project name* do Docker Compose

Sintoma: o primeiro deploy de staging subia no ambiente de produção sem criar os containers novos — o healthcheck ficou 502 por mais de 2 minutos até dar timeout.

Diagnóstico: como os dois ambientes clonam o repo para diretórios chamados `/opt/fiap-esg/staging/app` e `/opt/fiap-esg/production/app`, o Docker Compose derivou o mesmo *project name* (`app`) para os dois, tratando-os como o mesmo projeto e reutilizando os containers de produção.

Solução: adicionar `name: fiap-esg-${APP_ENV:-local}` no topo do `docker-compose.yml`. Isso garante que cada ambiente seja um *project* distinto, com containers, rede e volume isolados.

Emissão de certificado TLS para dois subdomínios novos

Sintoma: o nginx não iniciava com os configs finais pois os caminhos de cert ainda não existiam.

Solução: aplicar configs HTTP stub primeiro (só porta 80, com `/.well-known/acme-challenge/`), rodar `certbot certonly --webroot` para os dois domínios em uma única chamada (gerando um cert SAN), e só então substituir pelos configs HTTPS finais.

Webhook compartilhado entre vários apps no VPS

O host já rodava outros 3 deploy webhooks (`deploy-assistente-sus`, `deploy-quando-ein`, `deploy-blog`). O padrão existente foi preservado: adicionamos 2 novos hooks em `/etc/webhook/hooks.json`, cada um validando um `X-Deploy-Token` independente, e reiniciamos o `webhook.service` para recarregar a configuração sem afetar os outros deploys.

Seed automático do MongoDB

O trabalho anterior do grupo já trazia o script JS de seed das 5 collections. Em vez de reescrevê-lo em Python, aproveitamos o comportamento nativo da imagem oficial do MongoDB: arquivos `.js` em `/docker-entrypoint-initdb.d/` são executados automaticamente via `mongosh` contra o database definido em `MONGO_INITDB_DATABASE` na primeira subida. Apenas removemos a diretiva `use <db>` (exclusiva do shell interativo) e mantivemos todos os `insertMany()`.

6. Tecnologias utilizadas

Camada	Ferramentas
Linguagem / Runtime	Python 3.12
Framework Web	FastAPI + Uvicorn
Validação / DTOs	Pydantic v2 + pydantic-settings
Banco de dados	MongoDB 7 (driver assíncrono Motor)
Testes	pytest + pytest-asyncio + httpx + mongomock-motor
Qualidade	Ruff (lint + format)
Containerização	Docker multi-stage + Docker Compose v2
CI/CD	GitHub Actions + GitHub Environments
Webhook deploy	adnanh/webhook (binário Go, systemd)
Reverse proxy / TLS	nginx + Let's Encrypt (certbot)
Hospedagem	Hetzner Cloud CX33 (Ubuntu 24.04) — `pedrots.com.br`

7. Checklist de Entrega

Item	OK
Projeto compactado em .ZIP com estrutura organizada	✓
Dockerfile funcional (multi-stage, não-root, healthcheck)	✓
docker-compose.yml (api + mongo, volume, rede, env vars)	✓
Pipeline com etapas de build, teste e deploy	✓
README.md com instruções e prints	✓
Documentação técnica com evidências (este PDF)	✓
Deploy realizado em staging (`esg-staging.pedrots.com.br`)	✓
Deploy realizado em produção (`esg.pedrots.com.br`)	✓